Q21: How Do Different Networks and Learned RL Policies Combine in One System?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) name the five recurring composition patterns (pipeline, ensemble, mixture-of-experts, options/hierarchy, LLM-as-orchestrator) and give one deployed example of each, (ii) argue when composition beats monolithic end-to-end training and when it loses, (iii) explain the interface problem — what actually travels between learned components — and compare natural-language, structured-embedding, and programmatic-API interfaces, and (iv) apply a three-axis design heuristic (latency, trainability of joints, observability) to a concrete system proposal.

1 Why composition is the default, not the exception

No serious deployed learning system is a single network trained on a single loss. A self-driving stack runs perception, prediction, and planning networks in sequence; a frontier LLM agent calls tools; a robot kitchen assistant scores learned skills with a language model. The survey question behind this module asks *why* practitioners keep building systems out of multiple learned components, and what the design intuition is. The short answer: composition buys engineering properties — reusability, debuggability, heterogeneous-data compatibility — at the price of a harder learning problem at the seams. The rest of the note makes that trade precise. One scoping caveat: "composition" here means *distinct learned or algorithmic components wired together at run time*, not the internal layering of one jointly trained network, whose "joints" are not interfaces in the systems sense below.

2 Five composition patterns

Pipeline (sequential). Component A's output feeds component B: perception → prediction → planning in an autonomous vehicle, or an LLM that produces a plan consumed by an executor. Pipelines are the oldest pattern and the easiest to deploy incrementally. Their signature failure is *error propagation*: an upstream mistake becomes downstream ground truth, and B cannot ask A to reconsider. The modular perception–control stacks that end-to-end visuomotor policies were compared against are classic instances; Levine et al. pose the joint-vs-separate training question directly [7].

Ensemble (parallel voting). Several models see the same input and their outputs are averaged or voted. Ensembles reduce variance and are the cheapest robustness lever available, but every member runs on every input, so cost scales linearly and members trained on the same data make

correlated errors. Ensembles differ from mixtures (below) in that *all* components fire; there is no router.

Mixture-of-experts (gated routing). A learned *router* (the "gating network") inspects the input and activates one or a few of many *experts*. The idea dates to the 1990s: Jacobs, Jordan, Nowlan, and Hinton showed a gating network can learn to allocate training cases to expert sub-networks, localizing weight updates to the responsible expert [6]. Shazeer et al. revived it at scale with the sparsely-gated MoE layer, where a noisy top-k router activates a few of 137B experts per token, keeping FLOPs constant while growing parameters [10]; Switch Transformers simplified the router to top-1 and demonstrated stable training up to a trillion parameters [4]. MoE is composition *inside* a single training run — the joints are trained jointly — which is precisely why it avoids the interface problem that hurts cross-network composition, at the cost of requiring all experts to share one architecture, one data stream, and one loss.

Options and hierarchical RL (temporal composition). A high-level policy chooses among temporally extended sub-policies (*options*), each of which acts for many steps before returning control. Sutton, Precup, and Singh formalized options as closed-loop policies with initiation sets and termination conditions; a set of options over an MDP induces a semi-MDP at the higher level [11]. Dietterich's MAXQ decomposition showed how to decompose the value function across a task hierarchy, converging faster than flat Q-learning in the Taxi domain [3]; the option-critic architecture later learned both sub-policies and termination conditions end-to-end [2]. Hierarchy is composition *along the time axis*: the interface between levels is the option-selection event, and the central benefit is state abstraction — the high level can ignore everything the low level handles.

LLM-as-orchestrator (tool use). A language model maintains the reasoning state and calls learned or symbolic components as tools. ReAct interleaves natural-language reasoning traces with environment actions (search queries, Wikipedia API calls), beating pure chain-of-thought on multi-hop QA and imitation learning on ALFWorld/WebShop decision tasks [13]. Toolformer trains the model itself to emit API calls with arguments, self-supervised from whether a call improves loss on held-out text [8]. SayCan makes the orchestration probabilistic: an LLM scores the usefulness of each skill's *text description*, learned value functions score each skill's feasibility from the current state, and the product selects the next skill [1] — the options idea with the interface made of language. A different flavor is model-plus-search: MuZero composes a learned model (policy, value, reward heads) with classical tree search, where search is the orchestrator at decision time [9].

Pattern	Who selects	Interface medium	Runs per input
Pipeline	fixed wiring	upstream output	all, in order
Ensemble	nobody (all fire)	votes / average	all, in parallel
MoE	learned router	hidden activations	few (top-k)
Options/hierarchy	high-level policy	option choice + termination	one at a time
LLM orchestrator	LLM (prompted or trained)	natural language / API calls	as needed

Table 1: The five composition patterns; the interface column is the crux of Section 4.

3 When composition beats monolithic training — and when it loses

Composition wins when:

1. **Data heterogeneity.** Different components are trained on data that cannot be merged: a perception net on labeled images, a control policy on robot hours, an LLM on web text. A monolith would need all three in one stream with one loss; composition trains each where its data lives.
2. **Different observation rates.** A 10 Hz camera, a 100 Hz IMU, and a 0.1 Hz planner do not fit one training loop naturally. Options formalize this: the high level decides at option-boundary times, sub-policies at every step [11].
3. **Reusability.** A grasping skill trained once can be reused across tasks; SayCan adds a new capability by adding a skill description and a value function, without retraining the LLM [1]. A monolith must retrain to absorb each new task.
4. **Debuggability and unit testing.** Components can be evaluated in isolation, which matters enormously when the alternative is debugging an end-to-end policy's failure with no localizable fault. ReAct's explicit reasoning traces were valued specifically for interpretability and diagnosability [13].

Composition loses when:

1. **Compounding error.** If stage successes are even mildly dependent, a k-stage pipeline's end-to-end success rate falls roughly geometrically in k. This is the standard empirical argument for end-to-end training of perception-plus-control [7]: a joint policy can learn internal representations that no fixed intermediate interface would have preserved.
2. **Interface brittleness.** The consumer was trained on the producer's *past* output distribution. Retrain the producer and every downstream component silently degrades — there is no type system for embeddings. This is why MoE keeps its experts behind a jointly trained router rather than a frozen one [10].
3. **Credit assignment across components.** When the whole system fails, which component was wrong? In a pipeline the gradient simply does not flow backward across the seam (unless the interface is differentiable); in hierarchical RL, credit for a delayed reward must be split across the options that ran, which MAXQ handles only under a given, correct decomposition [3]. Ensembles dodge this by averaging; orchestrators dodge it only if the tool reports success/failure, as SayCan's value functions do [1].

The honest summary: composition trades *learning-theoretic optimality* (joint training sees all the data through one differentiable path) for *engineering scalability* (independent training, reuse, testing). Neither dominates; the choice is empirical and task-dependent.

4 The interface problem

Once you compose, something must cross each seam. Three regimes, in decreasing order of bandwidth and increasing order of human readability:

1. **Structured embeddings (vectors).** Maximum bandwidth, fully differentiable, cheap — but uninterpretable and unstable: the "protocol" is whatever the producer's weights happen

to emit, so any producer update is a breaking change with no error message. Suited to joints you intend to train together (MoE experts behind one router [10]) or fine-tune jointly (soft Q-composition adds Q-functions trained for different rewards, with a regret bound on the composed policy [5]).

2. **Programmatic APIs (typed calls).** Medium bandwidth, checked at the boundary: schemas, arguments, return values. Toolformer's calculator/calendar calls are the clean example [8]. The API is a contract the orchestrator can be trained against, and malformed calls fail loudly rather than silently. The cost is that someone must design the contract, and the learned caller must be reliable enough to use it.
3. **Natural language.** Lowest effective bandwidth and lossy, but universal: any component that can read text can join, and new tools join by writing a description rather than a schema. SayCan grounds LLM planning by multiplying language-model scores with learned affordance values, nearly doubling task success versus the ungrounded LLM alone [1]. The risks are ambiguity and hallucination, which is exactly why SayCan needed the value-function check.

A useful mental model: the interface choice sets *where gradient can flow*. Embeddings admit end-to-end fine-tuning; APIs admit reinforcement of the calling policy but not of the callee; language admits almost nothing to flow and instead relies on in-context generalization. Chain-of-thought prompting shows how far pure-language composition can go with no training at all [12] — and its brittleness at small scale shows the cost.

5 Design intuition: three axes

When you face a new system design, place it on three axes:

- **Latency.** How much added decision time can the task tolerate per seam? Pipelines add serial delay; ensembles run parallel (cost, not latency); MoE adds only the router forward pass [10]; an LLM orchestrator adds a full model generation per step, which is why ReAct-style agents are seconds-per-action, not milliseconds [13]; MuZero-style search composes model rollouts inside a fixed simulation budget [9].
- **Trainability of the joints.** Do you need gradients to cross the seams? If yes, the interface must be differentiable tensors (embeddings) and you are doing modular end-to-end training. If no, you can use discrete choices (routing, option selection, tool calls) and train components separately — accepting the credit-assignment burden.
- **Observability.** When the system fails at 3 a.m., can you localize the fault? Language interfaces log readable traces [13]; typed APIs fail with exceptions; embedding seams fail silently. Observability is why production systems often keep a language or API layer even when a learned embedding would be marginally more accurate.

These axes conflict: the highest-bandwidth joint (embeddings) is the least observable; the most observable (language) is the slowest and lossiest. Every real system is a negotiated position among the three, and the negotiation — not any single pattern — is the design skill this module wants to convey.

The toy demo. The module's code (`code/moe_vs_monolith.py`) trains a 4-expert mixture with a learned router on clustered synthetic data against a network with slightly *more* parameters (316 vs. 180, conservative for the mixture). The router learns to send each cluster to a specialist expert (100% routing accuracy, a permutation of the latent clusters) and the composed system reaches lower

held-out loss than the monolith (0.029 vs. 0.066); disable the router and route at random, and the same experts' loss explodes to 10.2. *Composition pays when the input distribution is heterogeneous and the router can learn the decomposition; the advantage is the interface, not the parameter count.*

Summary in one sentence

Composition buys reusability, heterogeneous-data compatibility, and debuggability at the price of compounding error, brittle interfaces, and cross-component credit assignment; the designer's real decision is not which pattern to pick but where on the latency / trainability / observability surface each seam should sit.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[2] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pages 1674–1680, 2017.

[3] Thomas G. Dietterich. Hierarchical reinforcement learning with the maxq value function decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000.

[4] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[5] Tuomas Haarnoja, Vitchyr Pong, Aurick Zhou, Murtaza Dalal, Pieter Abbeel, and Sergey Levine. Composable deep reinforcement learning for robotic manipulation. In *IEEE International Conference on Robotics and Automation*, pages 6284–6291, 2018.

[6] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[7] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[9] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[10] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[11] Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

[12] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.